



Objetivos:

- Conocer los Software a utilizar en el taller (Laravel).
- Completar la instalación de los diferentes programas y verificar el correcto funcionamiento.

Conceptos: Introducción a HTML. Arquitectura Cliente-Servidor. Patrón de diseño Modelo-Vista-Controlador (MVC). Framework PHP: Laravel.

Laravel es un framework de desarrollo web en PHP que proporciona herramientas y estructura para crear aplicaciones web utilizando el patrón arquitectónico Modelo-Vista-Controlador (MVC). Su objetivo es simplificar las tareas comunes utilizadas en el desarrollo web, como el enrutamiento, las plantillas, la autenticación y las interacciones con bases de datos, proporcionando una sintaxis limpia y expresiva. Laravel hace énfasis en la experiencia del desarrollador y promueve el desarrollo rápido de aplicaciones.

Requisitos previos:

Para comenzar a desarrollar con Laravel necesitamos crear nuestro entorno de desarrollo, se requieren las herramientas mencionadas en la Tabla 1.

¡Los instalaremos más adelante!

Herramienta	Versión Requerida
PHP	8.2 o superior
Composer	2.x (último estable)
Node.js / NPM	18.x o superior (opcional para frontend)
MySQL / SQLite	Cualquier versión reciente
Editor de código	VS Code recomendado con extensión Laravel

Tabla 1 - Pre-requisitos

a) Descargar e instalar el entorno de desarrollo **Laravel Herd**.

Para Windows: <https://herd.laravel.com/windows>

Para Mac OS: <https://herd.laravel.com>

Laravel Herd es un entorno de desarrollo nativo para Laravel que Incluye todo lo necesario para empezar a desarrollar con Laravel, incluyendo PHP y Nginx.



⚠ Usuarios de Linux ⚠

Laravel Herd no está disponible para sistemas operativos Linux.
Si utilizas alguna distribución de Linux (Ubuntu, Debian, Mint, etc.) deberás instalar los paquetes y servicios necesarios ejecutando el siguiente comando en la terminal:

```
/bin/bash -c "$(curl -fsSL https://php.new/install/linux/8.4)"
```

Una vez instalada la aplicación Herd verás una interfaz similar a la siguiente (Fig. 1)

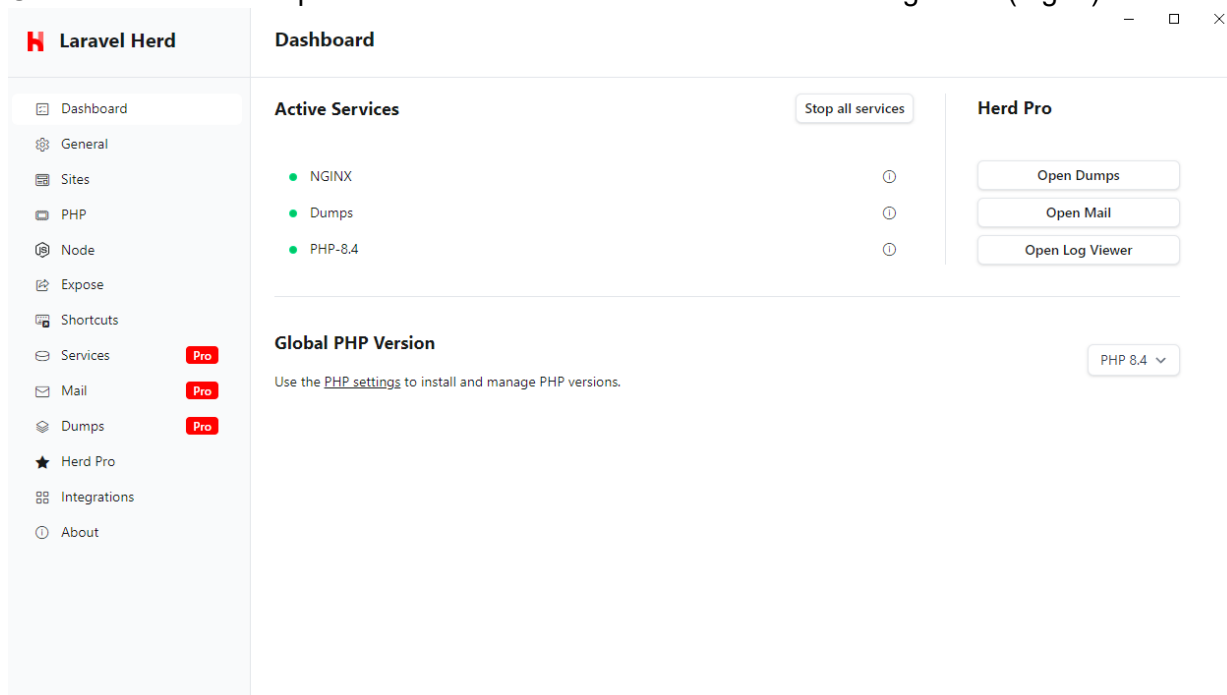


Fig. 1 - Laravel Herd

En el menú lateral izquierdo están los accesos rápidos a configuraciones y herramientas:

- **Dashboard:** Presenta el estado general de los servicios requeridos para desarrollar y desplegar en un entorno de desarrollo tus aplicaciones Laravel. Verifica que los servicios NGINX y PHP estén habilitados y corriendo (en verde).
- **General:** Permite cambiar ajustes por defecto como por ejemplo la carpeta donde se alojarán los proyectos, el IDE por defecto que se utilizará para escribir código, etc.
- **Sites:** Aquí podrás crear, importar y ver tus proyectos Laravel.



b) Descargar e instalar un editor de código fuente.

Para el desarrollo del proyecto necesitarás un editor de código, puedes utilizar el que prefieras, si aún no tienes definido cual utilizar te sugerimos alguno de los siguientes:

- **Visual Studio Code: el estándar actual, equilibrio entre potencia y ecosistema.**
 - Open Source.
 - Desarrollado por Microsoft.
 - Tiene Git integrado por defecto.
 - Tiene disponible el asistente IA Copilot de Github, útil para el autocompletado de código.
 - Existen muchos plugins disponibles para agregar funcionalidades.
 - Puedes descargarlo gratis en: <https://code.visualstudio.com/Download>
- **Sublime Text: el más rápido, liviano y minimalista.**
 - Es uno de los más livianos y rápidos.
 - Tiene Git integrado por defecto.
 - Cuenta con un ecosistema amplio de plugins.
 - Puedes descargarlo gratis en: <https://www.sublimetext.com/download>
- **Cursor: desarrollo asistido por IA. No recomendado para iniciar, ya que la IA puede modificar archivos y carpetas de forma autónoma.**
 - Es un entorno de desarrollo integrado (IDE) basado en inteligencia artificial, diseñado para acelerar la programación mediante la automatización y la generación de código.
 - Es un software de suscripción paga, pero que cuenta con una capa gratuita completamente funcional.
 - Es una bifurcación de Visual Studio Code y por lo tanto se pueden utilizar los plugins de VS Code.
 - Integra IA para editar, revisar y entender proyectos completos.
 - Puedes descargarlo gratis en: <https://cursor.com/es/download>



Actividades Prácticas

Actividad 1: Crea tu primer proyecto con Laravel

- 1) Abre la aplicación Laravel Herd y dirígete a la sección **Sites**, veras una pantalla similar a la que se muestra a continuación (Fig. 2). Luego presiona el botón **Add Site** para crear un nuevo proyecto Laravel.

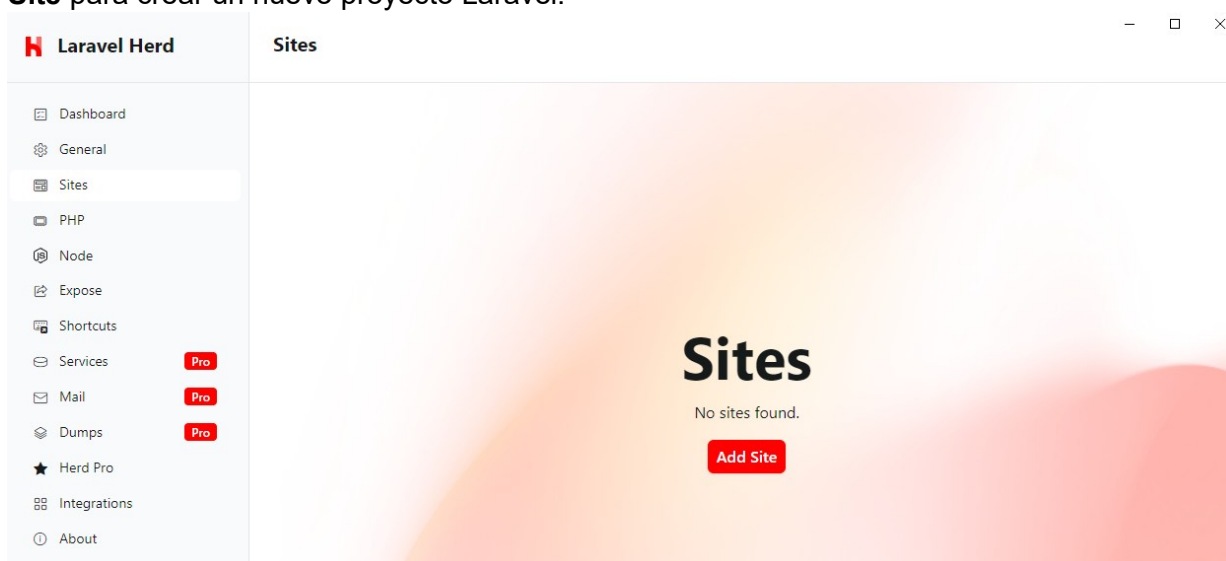


Fig. 2 - Sites

- 2) Selecciona **New Laravel Project**
- 3) Ahora selecciona la opción **No Start Kit**

Los Starter Kits proporcionan una estructura preconfigurada para las nuevas aplicaciones Laravel, ofreciendo una base con características comunes como autenticación, componentes de interfaz de usuario y estilos básicos ya configurados. Esto te permite comenzar rápidamente a crear funcionalidades específicas para cada aplicación sin tener que configurar estos elementos fundamentales desde cero. Agilizan el proceso de desarrollo inicial y promueven la coherencia entre los proyectos.

Más adelante veremos los starter kits, no te preocupes por esto ahora.



- 4) Por último solo queda definir el nombre del nuevo proyecto Laravel. Puedes utilizar el nombre “**primer_proyecto_laravel**” como en el ejemplo a continuación (Fig. 3) y esperamos que finalice la creación y configuración del proyecto nuevo.

Create New Site

Project Name:

primer_proyecto_laravel

Testing Framework:

Pest

Install Laravel Boost:

☒

Initialize a git repository:

☐

Target Location:

C:\Users\alons\Herd

Your project will be created in this folder.

Previous

Next

Fig. 3- Nuevo proyecto



- 5) Una vez finalizada la creación veremos una pantalla similar a la siguiente (Fig. 4). Si presionamos el botón **Open in Browser** se abra el navegador web y veremos la página de inicio de laravel, esto significa que el proyecto se creo correctamente (Fig. 5)

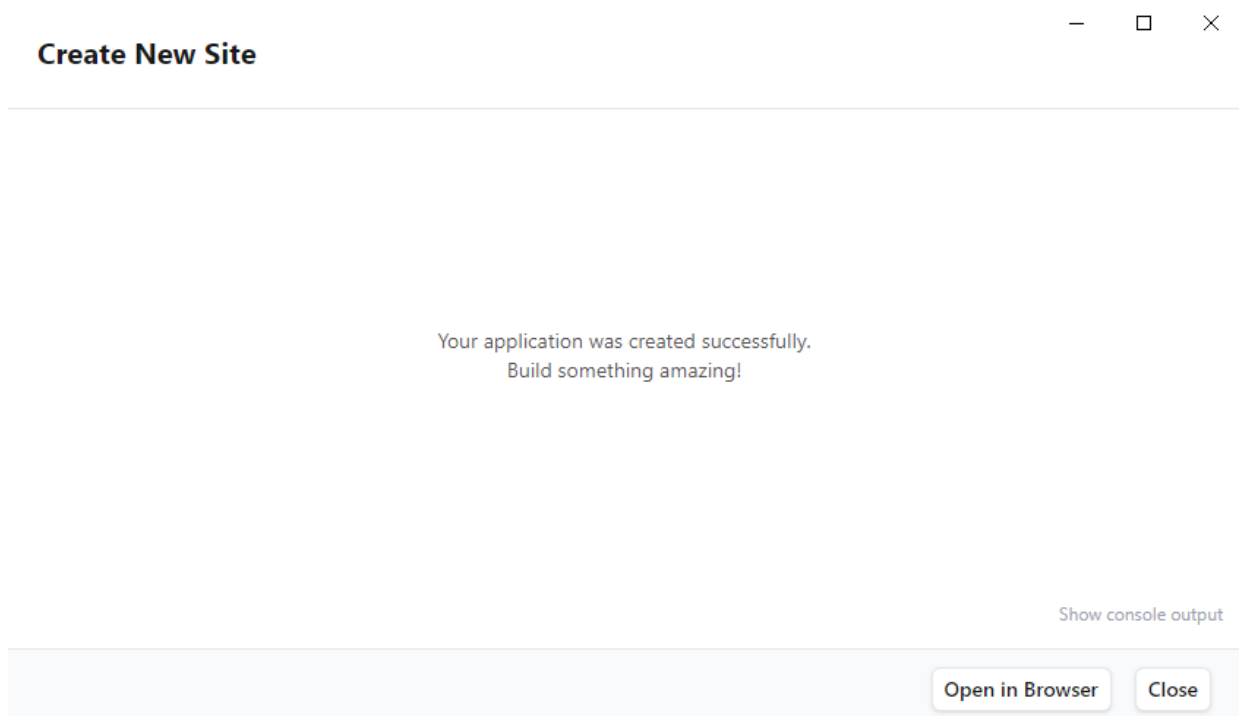


Fig. 4 - Proyecto creado

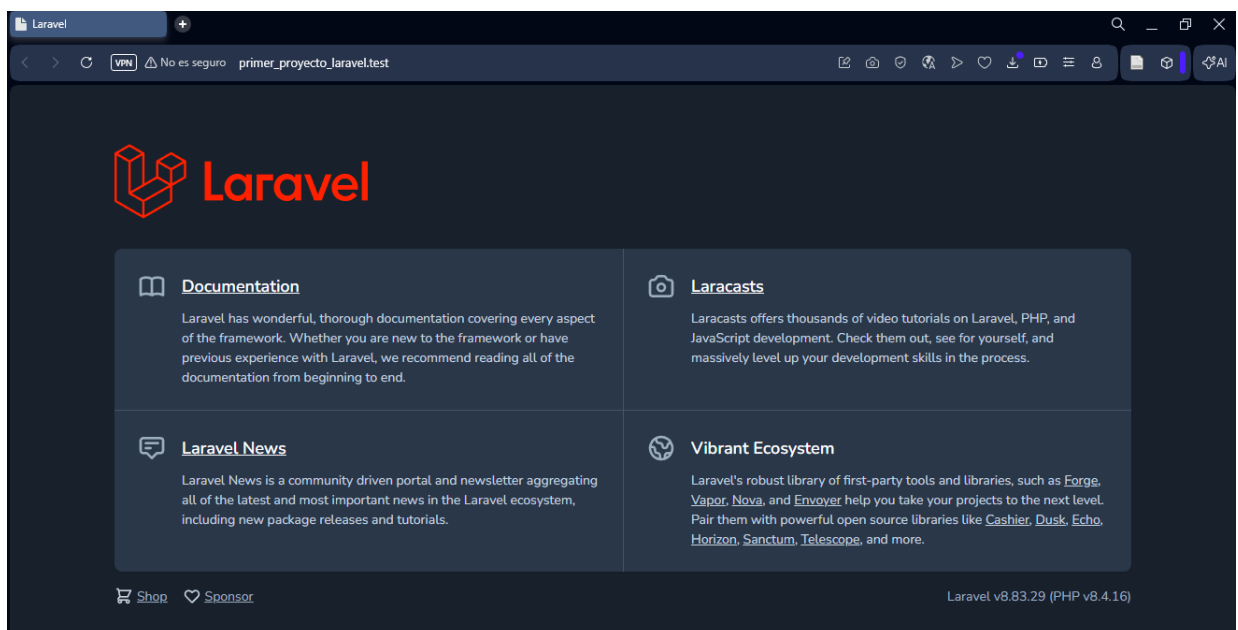


Fig. 5 - Vista inicial



- 6) Ahora de vuelta en la sección **Sites** de Laravel Herd se mostrará el proyecto creado en los pasos anteriores junto con accesos directos a la terminal, al IDE de desarrollo (VS Code, Sublime text, etc.), la ruta donde se creó el proyecto en nuestra computadora y la URL para acceder localmente a nuestro proyecto. En la Fig. 6 se muestra un ejemplo:

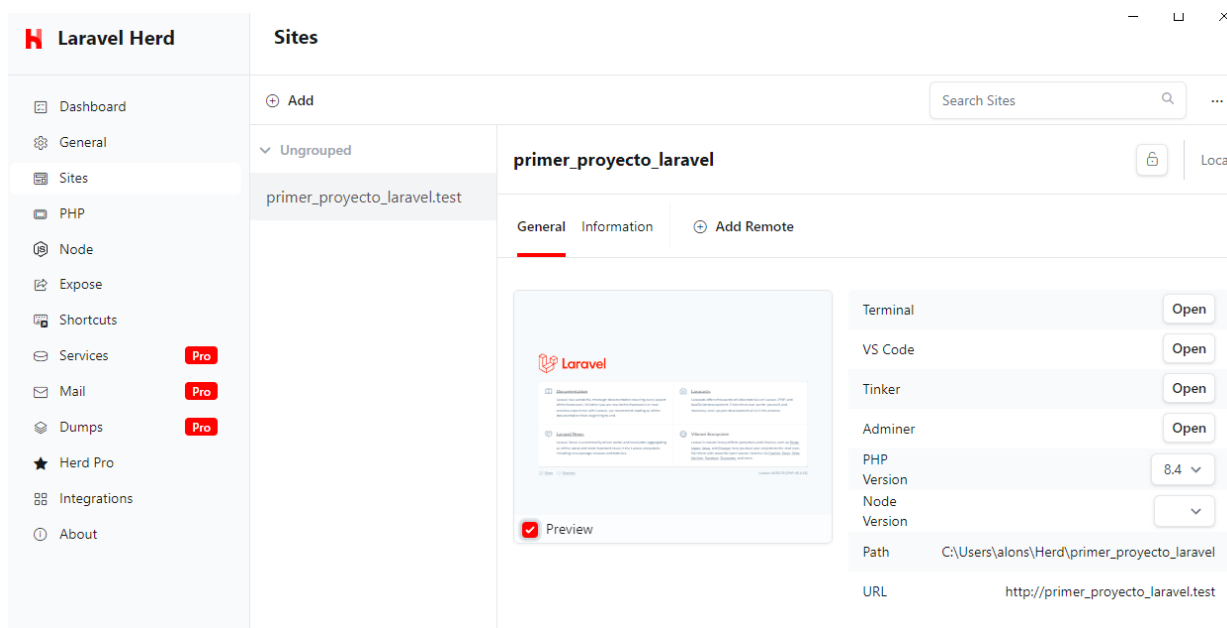


Fig. 6 - Lista de proyectos

- 7) Presionamos el botón **Open** que está a lado del IDE de desarrollo, en el ejemplo de la Fig. 6 el IDE de desarrollo es **VS Code**, para abrir el editor de texto y poder comenzar a trabajar en el proyecto.
- 8) En el editor de código podemos observar que el proyecto contiene carpetas y archivos (Fig. 7). ¿Qué son y para qué se utiliza cada carpeta y archivo?

Estructura del proyecto Laravel:

Veamos en detalle la estructura de Laravel de nuestro proyecto (Fig. 7).

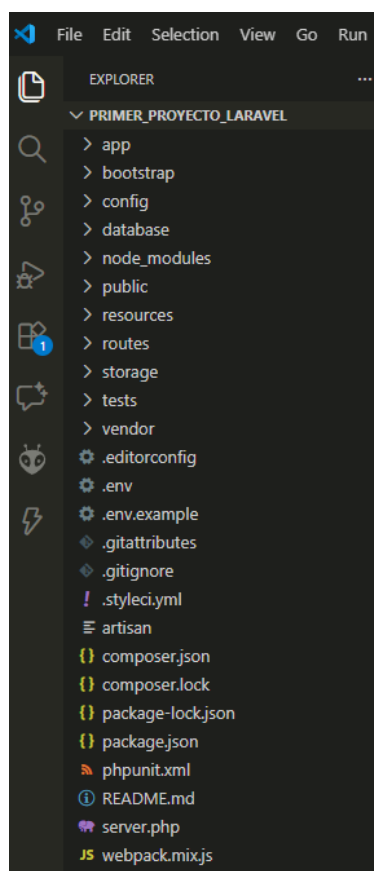


Fig. 7 - Estructura del proyecto Laravel

Directorio / Archivo	Propósito	¿Qué encontrarás?
app/	Núcleo de la aplicación	Modelos, Controladores, Middleware, etc.
app/Http/Controllers/	Controladores HTTP	Clases que manejan las requests
app/Models/	Modelos Eloquent	Clases que representan tablas de la BD
bootstrap/	Bootstrapping	Inicialización del framework
config/	Configuración	Archivos de configuración por módulo
database/	Base de datos	Migraciones, seeders y factories
public/	Raíz web pública	index.php, assets compilados
resources/	Recursos frontend	Vistas Blade, CSS, JS
routes/	Definición de rutas	web.php, api.php, console.php
storage/	Almacenamiento	Logs, cache, archivos subidos
.env	Variables de entorno	Configuración local (DB, mail, etc.)
artisan	CLI de Laravel	Punto de entrada de comandos Artisan



Actividad 2: Crea tu primera página web personalizada

Antes de comenzar debemos entender el funcionamiento del patrón de diseño Modelo-Vista-Controlador (MVC).

Flujo básico:

1. Usuario entra a una URL
- ↓
2. Ruta (routes/web.php)
- ↓
3. Vista (HTML)
- ↓
4. Página mostrada al usuario

Ruta: Define **qué hacer cuando se accede a una URL**

Vista: Es la **interfaz visual (HTML/CSS)** que ve el usuario

a) Crear la ruta

Las rutas en Laravel definen **qué URL responde a qué código**.

- a. Abrir el archivo: **routes/web.php**
- b. Agregar al final del archivo:

```
Route::get('/sobre-mi', function () {  
    return view('sobre-mi');  
});
```

Esto significa:

- Cuando alguien entra a **/sobre-mi**
- Laravel ejecuta la función
- y devuelve la **vista sobre-mi**

b) Crear la vista

- a. Crear el archivo: **resources/views/sobre-mi.blade.php**
Laravel usa el motor de plantillas Blade para generar HTML dinámico.
- b. Dentro del archivo debemos escribir el código HTML de nuestra nueva página.
A continuación hay un ejemplo, copia y pega este código y actualiza tus datos en él.

```
<!DOCTYPE html>  
<html>
```



```
<head>
<title>Sobre mí</title>

<style>
body {
font-family: Arial;
background-color: #f4f4f4;
margin: 40px;
}

.container {
background: white;
padding: 20px;
border-radius: 10px;
max-width: 600px;
}

h1 {
color: #333;
}

p {
line-height: 1.6;
}
</style>

</head>

<body>

<div class="container">

<h1>Sobre mí</h1>

<p><b>Nombre:</b> Juan Pérez</p>

<p><b>Edad:</b> 22 años</p>

<p><b>De dónde soy:</b> Corrientes, Argentina</p>

<p><b>Me gustaría trabajar en:</b> Desarrollo de software</p>

<p><b>Expectativas del curso:</b> Aprender a crear aplicaciones
web</p>

<p><b>Hobbies:</b> Programar, fútbol y videojuegos</p>

</div>

</body>
</html>
```



Etiquetas HTML utilizadas:

Etiqueta	Función
<html>	Contenedor de todo el documento
<head>	Información de la página
<title>	Título de la pestaña
<body>	Contenido visible
<div>	Contenedor de elementos
<h1>	Título principal
<p>	Párrafo
	Texto en negrita
<style>	Estilos CSS

- c. En el navegador introduce la URL de tu proyecto y agrega al final **/sobre-mi** para ver la nueva página creada. Si utilizaste el nombre de proyecto sugerido en esta guía deberías poder ver la nueva página con la siguiente URL:
http://primer_proyecto_laravel.test/sobre-mi

Recuerda siempre tener iniciado Laravel Herd y verificar que los servicios estén funcionando (en verde)

¡Felicitaciones!

🎉 Creaste tu primera página web con Laravel 🎉

Deberías ver una página similar a la que se muestra en la Fig. 8 si todo salió bien

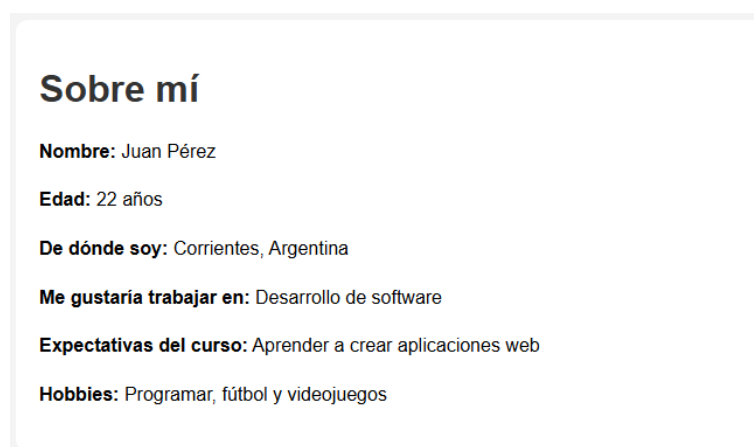


Fig. 8 - Página sobre mi



Conceptos clave:

Estructura del proyecto Laravel

Directorios importantes:

- **App:** Esta carpeta alberga la lógica central de tu aplicación. Contiene el código que define el comportamiento de tu aplicación, incluyendo modelos, controladores, middleware, services y otras clases personalizadas. Este directorio está estructurado para facilitar la organización y el mantenimiento, lo que simplifica la gestión y el escalado de tu aplicación a medida que crece.
- **Config:** Los archivos de configuración en Laravel te permiten administrar los ajustes de tu aplicación de forma centralizada y organizada. Estos archivos almacenan pares clave-valor que definen diversos aspectos del comportamiento de tu aplicación, como las conexiones a bases de datos, la configuración del correo electrónico y los parámetros específicos de la aplicación. Mediante el uso de archivos de configuración, puedes modificar fácilmente los ajustes de tu aplicación sin alterar directamente el código, lo que la hace más flexible y fácil de mantener en diferentes entornos.
- **Public:** Sirve como raíz de documentos para tu aplicación Laravel. Es el único directorio al que se debe poder acceder directamente desde la web. Este directorio contiene el archivo index.php, que es el punto de entrada para todas las solicitudes HTTP que ingresan a tu aplicación, así como recursos como imágenes, CSS y archivos JavaScript.
- **Routes:** El directorio de rutas en Laravel contiene todas las definiciones de rutas para tu aplicación. Estos archivos le indican a Laravel cómo responder a diferentes solicitudes HTTP (como GET, POST, PUT, DELETE) para URL específicas. Cada archivo suele definir un conjunto de rutas, asignando una URL a una acción o cierre específico del controlador que gestionará la solicitud y devolverá una respuesta.
- **Bootstrap:** Bootstrap en un proyecto Laravel es el punto inicial en el que el framework inicia su proceso de ejecución. Implica configurar el entorno, cargar configuraciones, registrar proveedores de servicios y gestionar excepciones. Este proceso garantiza que todos los componentes y dependencias necesarios estén disponibles y correctamente configurados antes de que la aplicación comience a procesar solicitudes.
- **Database:** El directorio de la base de datos en un proyecto Laravel alberga todos los archivos relacionados con las interacciones de la base de datos de tu aplicación. Esto incluye migrations, que definen la estructura de las tablas de tu base de datos; seeders, que llenan tu base de datos con datos iniciales; factories, que generan datos falsos para pruebas; y cualquier lógica personalizada relacionada con la base de datos que puedas implementar. Sirve como una ubicación central para administrar y controlar las versiones de tu esquema de base de datos y datos iniciales.
- **Resources:** El directorio de recursos de un proyecto Laravel es donde se almacenan los activos sin compilar de la aplicación. Esto incluye elementos como las vistas (plantillas HTML), los archivos de idioma, CSS, JavaScript e imágenes. Estos activos



suelen ser procesados por herramientas como Webpack o Vite antes de ser servidos al usuario.

Algunos de los archivos que estan en la raiz del proyecto:

- **.env:** Es un archivo de texto plano que utilizamos para gestionar credenciales y configuraciones específicas del entorno (desarrollo, producción) sin exponerlas en el repositorio de código. Este archivo contiene todas las variables de entorno de nuestro proyecto y no debemos nunca subirla a repositorios o hacerla pública.

Artisan

Es la interfaz de línea de comandos (CLI) incluida en Laravel que automatiza tareas repetitivas y gestiona el proyecto. Se utiliza ejecutando comandos php artisan en la terminal para crear controladores, migraciones, modelos, limpiar caché y gestionar la base de datos, acelerando el desarrollo.

Veamos algunos comandos de artisan que utilizaremos más adelante:

Comando	Descripción
php artisan serve	Inicia el servidor de desarrollo en puerto 8000
php artisan list	Lista todos los comandos disponibles
php artisan make:controller NombreController	Crea un nuevo controlador
php artisan make:model Nombre -m	Crea un modelo con su migración
php artisan migrate	Ejecuta las migraciones pendientes
php artisan migrate:fresh --seed	Reinicia la BD y corre los seeders
php artisan route:list	Muestra todas las rutas registradas
php artisan tinker	Abre un REPL interactivo de PHP/Laravel
php artisan config:clear	Limpia la caché de configuración



Diccionario básico:

HTTP (HyperText Transfer Protocol)

Es el protocolo que usan los navegadores y servidores para comunicarse en la web.

Permite enviar **peticiones (requests)** desde un cliente (navegador) y recibir **respuestas (responses)** desde un servidor.

Ejemplo: cuando abris una página web, tu navegador envía una petición HTTP al servidor.

Métodos HTTP

GET

Solicita datos al servidor.

No debería modificar información.

Ejemplo:

GET /usuarios

Obtiene la lista de usuarios.

POST

Envía datos al servidor para **crear recursos**.

Ejemplo:

POST /usuarios

Crear un usuario nuevo.

PUT

Se utiliza para **actualizar completamente** un recurso existente.

Ejemplo:

PUT /usuarios/5

Actualiza el usuario con ID 5.

Códigos HTTP

Son códigos que el servidor devuelve indicando el resultado de una petición.

Ejemplos comunes:

Código	Significado
---------------	--------------------

200	OK (todo correcto)
201	Recurso creado
400	Error en la petición
401	No autorizado
404	Recurso no encontrado
500	Error interno del servidor



HTML (HyperText Markup Language)

Es el lenguaje que define la **estructura de una página web**.

Ejemplo:

```
<h1>Hola mundo</h1>
<p>Este es un párrafo</p>
```

PHP

Lenguaje de programación que se ejecuta en el **servidor**.
Laravel está escrito en PHP.

Permite:

- Procesar datos
- Conectarse a bases de datos
- Generar HTML dinámicamente

Patrón MVC (Model – View – Controller)

Es un patrón de arquitectura que separa responsabilidades.

Componente Función

Model	Maneja los datos y la base de datos
View	Muestra la información al usuario
Controller	Recibe peticiones y coordina la lógica

Laravel usa este patrón.

Arquitectura Cliente-Servidor

Modelo donde:

- **Cliente** → hace peticiones (navegador, app)
- **Servidor** → procesa y responde

Nginx

Servidor web que:

- recibe las peticiones HTTP
- sirve archivos web
- redirige peticiones a aplicaciones como PHP/Laravel

Es una alternativa a Apache.

Composer

Gestor de **dependencias para PHP**.

Permite instalar librerías externas fácilmente.

Ejemplo:

```
composer install
```



Artisan

Herramienta de línea de comandos incluida en Laravel.

Permite:

- crear controladores
- ejecutar migraciones
- iniciar el servidor

Ejemplo:

php artisan serve

Dependencias

Librerías o paquetes que un proyecto necesita para funcionar.

Ejemplo:

- Laravel depende de varios paquetes PHP instalados con Composer.
-

Peticiones (request)

Es el mensaje que el cliente envía al servidor.

Incluye:

- método HTTP
 - URL
 - parámetros
 - datos
-

Querys

Consulta enviada a una **base de datos** para obtener o modificar información.

Ejemplo SQL:

```
SELECT * FROM usuarios
```

Obtiene todos los datos de todos los usuarios registrados en la base de datos

Rutas

Definen **qué código se ejecuta cuando se accede a una URL**.

Ejemplo en Laravel:

```
Route::get('/usuarios', [UserController::class, 'index']);
```

Controladores

Clases que reciben las peticiones del usuario y ejecutan la lógica necesaria.

Ejemplo:

- obtener datos
- validar información



- devolver una vista
-

Middleware

Filtros que se ejecutan **antes o después de una petición.**

Se usan para:

- autenticación
 - permisos
 - logs
 - seguridad
-

Frontend

Parte de la aplicación que **ve e interactúa el usuario.**

Tecnologías comunes:

- HTML
 - CSS
 - JavaScript
-

Backend

Parte de la aplicación que **corre en el servidor.**

Se encarga de:

- lógica
- base de datos
- seguridad
- APIs

Laravel es backend.

Base de datos

Sistema que almacena información estructurada.

Ejemplos:

- MySQL
- PostgreSQL
- MariaDB

Se usan para guardar datos como usuarios, productos o pedidos.

API REST (Representational State Transfer)

Es un estilo de arquitectura para crear APIs.

Características:

- usa HTTP
- usa URLs para recursos
- usa métodos HTTP (GET, POST, PUT, DELETE)
- devuelve generalmente JSON



Ejemplo:

GET /api/usuarios

CSS (Cascading Style Sheets)

Define la **apariciencia visual** de una página web.

Ejemplo:

```
h1 {  
  color: blue;  
}
```

Vue

Framework de **JavaScript** para crear interfaces de usuario.

Características:

- reactivo
 - fácil de integrar
 - muy usado con Laravel
-

React

Biblioteca de **JavaScript** creada por **Meta** para construir interfaces web.

Basada en **componentes reutilizables**.

Localhost

Nombre que se refiere a **la computadora local**.

Ejemplo:

http://localhost

Se usa para desarrollo local.

DNS (Domain Name System)

Traduce nombres de dominio a direcciones IP.

Ejemplo:

google.com → 142.250.x.x

Permite que los humanos usen nombres en lugar de IPs.



Referencias:

- [Guía oficial de Laravel 12.x](#)
- [Hoja de ruta para aprender Laravel](#)
- [Hoja de ruta para aprender PHP](#)
- [Guía oficial de Laravel Herd](#)

Enlaces de interés:

- **Fazt:** Es una plataforma de cursos gratuitos sobre desarrollo en general, web, frameworks, bases de datos, herramientas, IA, etc.
Podrías comenzar viendo los cursos sobre web: <https://fazt.dev/web>
- **Pildoras informáticas:** Es una plataforma de cursos gratuitos que tiene cursos de todo tipo, ideal para entender PHP, HTML, CSS y JavaScript.
Podes comenzar aquí y luego explorar más: <https://www.pildorasinformaticas.es/course-cat/programacion/>
- **Laravel learn:** Tutoriales oficiales de Laravel, sobre el framework y PHP.
<https://laravel.com/learn>
- **Codecourse:** Canal de YouTube sobre programación. Tiene muchos cursos de Laravel, pero en inglés. <https://www.youtube.com/@codecourse/courses>
- **Laravel Ole:** Comunidad en español sobre Laravel. Consejos, mejores prácticas, guías y novedades en español. <https://laravelole.es>
- **Laracast:** Series y podcast de Laravel. Laravel desde 0 y niveles más avanzados.
<https://laracasts.com/series/laravel-from-scratch-2026>